

NYCU-EE IC LAB – Autumn 2025

Lab08

Design: Self-Attention with Determinant (SAD)

Data Preparation

1. Extract test data from TA's directory:

```
% tar xvf ~iclabTA01/Lab08_2025A.tar
```

2. The extracted LAB directory contains:

I. EXERCISE_wocg

a. 00_TESTBED

b. 01_RTL

c. 02_SYN

d. 03_GATE_SIM

II. EXERCISE

a. 00_TESTBED

b. 01_RTL

c. 02_SYN

d. 03_GATE_SIM

e. 04_PTPX

g. 09_SUBMIT

III. JG

Design Description

Self-attention is a mechanism used in neural networks, particularly in transformer models, to determine the importance of each part of an input sequence when making predictions. Scaled dot-product attention projects input matrices (Query (Q), Key (K), and Value (V)), computes scaled dot-product attention scores, applies the Softmax function to these scores, and generates a context matrix, allowing the model to focus on relevant parts of the input sequence.

In this Lab, you'll be designing a simple hardware accelerator for Self-Attention. The accelerator should efficiently handle matrix operations for input sizes of 1x8, 4x8, and 8x8. The results of the accelerator will multiply a factor from the 4x4 matrix determinant as outputs. Finally, designer should save the power of this accelerator with clock gating.

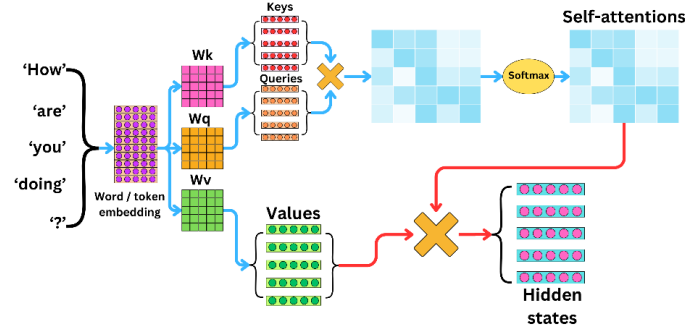


Figure 1. The mechanism of self-attentions.

The accelerator receives few input signals: “in_valid”, “in_data1”, “T”, “in_data2”, “w_Q”, “w_K”, and “w_V”. When “in_valid” is high, it indicates there is at least one valid input signal. The “T” signal is valid in the first cycle. At the same time, the 6-bit signed “in_data1” signal is valid for first 16 consecutive cycles for 4x4 matrix determinant; the 8-bit signed “in_data2” signal is valid for first Tx8 consecutive cycles for tokens of self-attention. The three weights are 8-bit signed signals. The input of “w_Q” is valid in the first 64 cycles. Next, the input of “w_K” is followed for next 64 cycles. Finally, the input of “w_V” is valid in the last 64 cycles. Therefore, the “in_valid” signal is high for 64x3 = 192 consecutive cycles. Moreover, the signal of “in_data1”, “in_data2”, “w_Q”, “w_K”, and “w_V” are sent in raster scan order. Noted that when “in_valid” is low, all input signals mentioned above must be tie to unknown; however, when “in_valid” is high, these signals may be unknown in certain specific cycles.

Your accelerator outputs two signals, “out_valid” and “out_data”, for Tx8 consecutive cycles. When “out_valid” is high, it indicates the “out_data” signal is valid. The signal of “out_data” is the product of result of determinant and the ones of self-attention in raster scan order.

The following is the step of this accelerator:

Formula of QKV Generation:

$$Q = XW_Q, K = XW_K, V = XW_V$$

Formula of 1st matrix multiplication:

$$A = QK^T$$

Formula of Scale:

For simplicity, the scaling factor ($\sqrt{d_{model}}$) is a constant, 3.

$$Scale = \frac{QK^T}{3}$$

Formula of ReLU:

For simplicity, the Softmax function has been replaced with ReLU:

$$\text{ReLU}(x_i) = \max(0, x_i)$$

$$S = \text{ReLU}\left(\frac{QK^T}{3}\right)$$

Formula of 2nd matrix multiplication:

$$P = \text{ReLU}\left(\frac{QK^T}{3}\right)V$$

Formula of determinant:

$$D = \text{Det}(Y_{4 \times 4})$$

Formula of 3rd matrix multiplication:

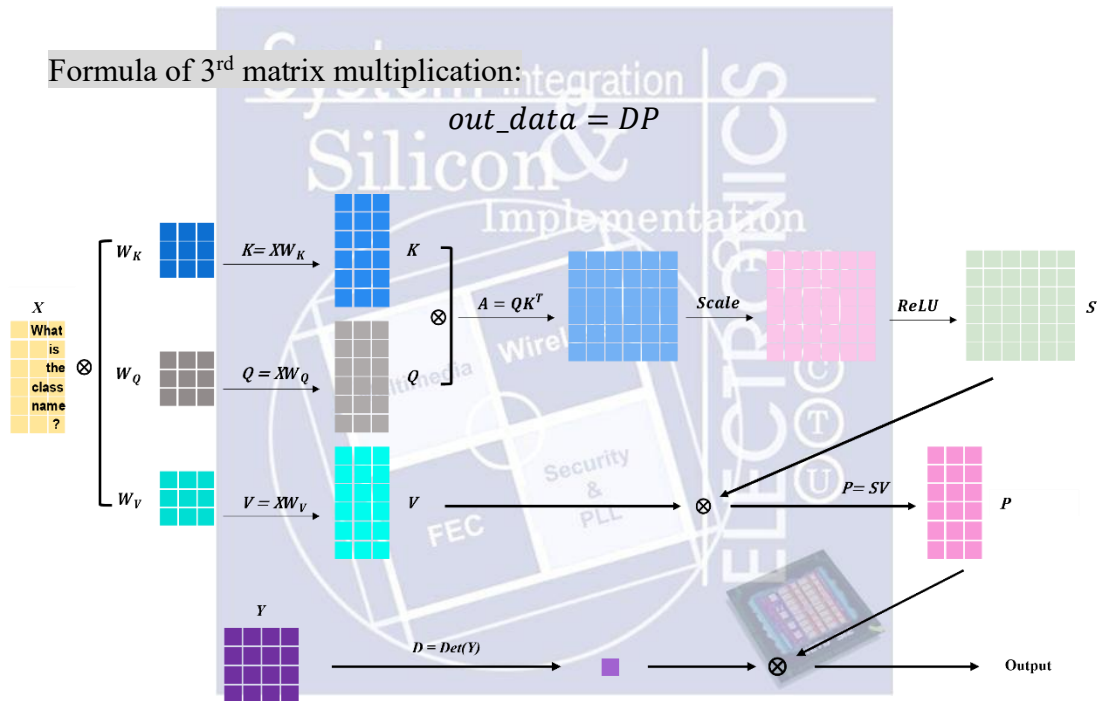


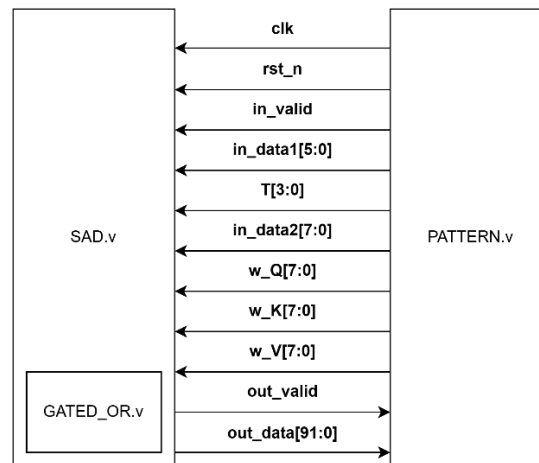
Fig2. The step of the accelerator in this Lab08.

Original Formula of Scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_{\text{model}}}}\right)V$$

Note: All of the computations mentioned above must ensure that their process and results do not overflow.

System Architecture



Inputs

1. When input signal **in_valid** is **high**, at least one input signal is valid.
2. When input signal **in_valid** is **low**, the input signal **in_data1**, **T**, **in_data2**, **w_Q**, **w_K**, and **w_V** must be tied to unknown (X).
3. All input signals are synchronized at the clock **negative** edge.
4. This design involves in **signed operation**.
5. There is **only 1 reset** before the first pattern, thus, your design must be able to reset automatically.
6. The next input signal will come in 2-5 cycles after **out_valid** is pulled down.
7. The input **cg_en** is in SAD.v for enable signal of clock gating.

Name	Width	Functional Description
clk	1	Clock for SDA
rst_n	1	Asynchronous active-low reset
cg_en	1	If cg_en is high, the series processing blocks should execute clock gating. Otherwise, if cg_en is low, the processing blocks follow clk .
in_valid	1	When in_valid is high, the input signals are valid.
in_data1	6	in_data1 is valid when in_valid is high. The in_data1 will be given in 16 consecutive cycles in raster scan order. The values are signed integers. (-32~31)
T	4	The input sequence length T will be given in the first cycle when in_valid is high. To be simpler, T = 1, 4, 8

in_data2	8	in_data2 is valid when in_valid is high. The in_data2 will be given in Txd_{model} consecutive cycles in raster scan order. To be simpler, d_{model} always equals 8. The values are signed integers. (-128~127)
w_Q	8	w_Q is valid when in_valid is high. The w_Q will be given in 64 consecutive cycles in raster scan order. The values are signed integers. (-128~127)
w_K	8	w_K is valid when in_valid is high. The w_K will be given in 64 consecutive cycles in raster scan order. The values are signed integers. (-128~127)
w_V	8	w_V is valid when in_valid is high. The w_V will be given in 64 consecutive cycles in raster scan order. The values are signed integers. (-128~127)

Table 1. Input signal information

Outputs

1. When output signal **out_valid** is **high**, the output signal **out** must be delivered in **raster scan order** for Txd_{model} consecutive cycles.
2. When output signal **out_valid** is **low**, the output signal **out** is tied to logic-low.
3. All outputs must be synchronized at the clock **positive** edge.
4. The output signal **out_valid** and **out_data** should be logic-low after initial reset.

Name	Width	Functional Description
out_valid	1	When out_valid is high, the output out is valid.
out_data	92	The results output in raster scan order. out_data should be given in Txd_{model} cycles. To be simpler, d_{model} always equals 8.

Table 2. Output signal information

Specifications

1. Top module name: **SAD** (File name: **SAD_wocg.v/SAD.v**)
2. It is **asynchronous** reset and **active-low** architecture.

3. The reset signal (**rst_n**) would be given only once at the beginning of simulation.
4. Some data are in **signed** format.
5. The output signal **out_data** should be logic-low when the output signal **out_valid** isn't high.
6. When the output signal **out_valid** is high, the output signal **out_data** must be correct, and both should be high for Txd_{model} consecutive cycles.
7. The total cell area should not larger than **2,000,000 μm^2** .
8. The latency of design in each pattern should not be larger than **1000 cycles**. The latency is the clock cycle between the falling edge of the input signal **in_valid** and the rising edge of the output signal **out_valid**.
9. The **out_valid** cannot overlap with **in_valid**.
10. The maximum clock period is set to **50 ns**, and **you cannot determine the clock period by yourself**.
11. The input delay and the output delay should be **half** of the clock period, i.e., the input/output delay is set to $0.5 \times (\text{clock period})$.
For example, if clock period is 50 ns, the input/output delay will be 25 ns.
12. The output loading is set to 0.05.
13. **Hardcoding answers** or using a **lookup table** based on the test case index is prohibited; your design must compute outputs correctly based on the actual inputs. You will get 0 point if you use this method, **no second chance**.
14. **Synopsys' IP** and **memory** is prohibited in this lab.
15. Only comments in English are allowed.
16. The synthesis result of data type cannot include any **LATCH except for clock gating latch**.
17. System Verilog syntax—if your demo fails due to any of the issues, TAs will not be held responsible.
18. All data flip-flops (D flip-flop/DFF) in your design must be **reset** after the reset signal is asserted.
19. The slack in the end of SAD.timing should be **non-negative** and the result should be **MET**.
20. Note that maximum synthesis time could not be over **1 hours**.
21. The gate-level simulation cannot include **any warning of timing violations**.
22. No **ERROR** is allowed in simulation/synthesis log.
23. Don't use any wire / reg / submodule name called "**error**", "**congratulation**", "**latch**" or "**fail**", or you will fail the lab.
24. The output signal **out_valid** in SAD_wocg.v must be **the same** as the one in SAD.v.
25. The instance name of the clock gating cell (GATED_OR) should be

GATED_XXX (e.g., GATED_cnt).

```
GATED_OR GATED_out (
    .CLOCK(clk), .SLEEP_CTRL(G_sleep_out),
    .RST_N(rst_n), .CLOCK_GATED(G_clock_out)
);
```

26. Your design should have **at least 20% power reduction from cg_en-off to cg_en-on**, otherwise it will be regarded as failed. $\frac{P_{cg_enoff} - P_{cg_enon}}{P_{cg_enoff}} \geq 20\%$.

27. Power report position:

04_PTPX/Report/SAD_POWER or SAD_CG_POWER

- Example:

SA_POWER (w/o clock gating)					SA_CG_POWER (w/ clock gating)				
Power Group	Internal Power	Switching Power	Leakage Power	Total Power (%)	Power Group	Internal Power	Switching Power	Leakage Power	Total Power (%)
clock_network	4.948e-04	3.535e-03	5.135e-07	4.031e-03 (14.33%)	clock_network	8.113e-05	3.974e-04	3.722e-07	4.789e-04 (2.29%)
register	7.145e-03	3.389e-04	3.089e-05	7.515e-03 (26.72%)	register	3.357e-03	3.413e-04	3.073e-05	3.730e-03 (17.78%)
combinational	6.160e-03	0.0104	5.238e-05	0.0166 (58.95%)	combinational	6.231e-03	0.0105	5.245e-05	0.0168 (79.92%)
sequential	0.0000	0.0000	0.0000	0.0000 (0.00%)	sequential	0.0000	0.0000	0.0000	0.0000 (0.00%)
memory	0.0000	0.0000	0.0000	0.0000 (0.00%)	memory	0.0000	0.0000	0.0000	0.0000 (0.00%)
io_pad	0.0000	0.0000	0.0000	0.0000 (0.00%)	io_pad	0.0000	0.0000	0.0000	0.0000 (0.00%)
black_box	0.0000	0.0000	0.0000	0.0000 (0.00%)	black_box	0.0000	0.0000	0.0000	0.0000 (0.00%)
Net Switching Power	= 0.0142 (50.63%)				Net Switching Power	= 0.0112 (53.46%)			
Cell Internal Power	= 0.0138 (49.07%)				Cell Internal Power	= 9.670e-03 (46.14%)			
Cell Leakage Power	= 8.379e-05 (0.30%)				Cell Leakage Power	= 8.355e-05 (0.40%)			
Intrinsic Leakage	= 8.379e-05				Intrinsic Leakage	= 8.355e-05			
Gate Leakage	= 0.0000				Gate Leakage	= 0.0000			
Total Power	= 0.0281 (100.00%)				Total Power	= 0.0210 (100.00%)			
X Transition Power	= 2.011e-05				X Transition Power	= 2.010e-05			
Glitching Power	= 0.0000				Glitching Power	= 0.0000			
Peak Power	= 0.5187				Peak Power	= 0.4255			
Peak Time	= 7297050				Peak Time	= 3496154			

28. The performance is determined by **latency**, and **power**.

Grading Policy

- 70% for functional validity, 10% for JasperGold SEC_run1, and 20% for performance:

Score = functional validity (70%) + JasperGold SEC_run1 (10%) + performance (20%)

- The performance is determined by **latency**, and **power** of your design. The smaller the performance index is, the higher score you can get.

$$\text{Perf.} = \text{latency} \times \text{power}^2 \text{ (w/ CG)}$$

- Note that cycle period must be 50 ns, or you will fail this lab.**
- Note that latency must within 1,000 cycles, or you will fail this lab.**
- Note that cell area must within 2,000,000 μm^2 , or you will fail this lab.**
- Performance points can be obtained if and only if your design passes the TA's patterns.
- TA will demo your design with the TA's patterns including **the hidden ones**.
- The first ten submissions** of designs that pass the TA's pattern will receive a 5-

points bonus.

9. The successfully pass of JasperGold SEC_run2 will receive a 5-points bonus.
10. 2nd demo will only get **0.7x points**.
11. There is **no second chance** for JasperGold SEC check.
12. You will get **0 point** if you are found **plagiarism** at your code; **no second chance**.

Note

1. Please submit your files under 09_SUBMIT before 12:00 at noon:

Due Day: 1st Demo: 11/10 2nd Demo: 11/12

If uploaded files **violate the naming rule**, you will get **5 deduct points**

- In this lab, you cannot adjust your clock cycle time.
- After that, you should check the following files under

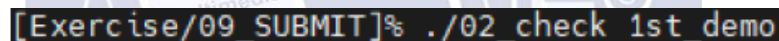
09_SUBMIT/Lab08_iclabXXX/

RTL design: **SAD_iclab???.v** and **SAD_wocg_iclab???.v**, (??? is your account number)

File list: **filelist_iclab???.f** (ex: **filelist_iclab999.f**)

If you miss any files on the list, you will fail this lab.

Then use the command like the figure below to check if the files are uploaded or not.



```
[Exercise/09_SUBMIT]% ./02_check 1st_demo
```

Example:

You will submit **SAD_iclab999.v**, **SAD_wocg_iclab999.v**, and **filelist_iclab999.f**

If the uploaded files violating the naming rule, you will get **5 deduct points**.

If you omit any file, you will fail demo.

2. Template folders and reference commands:

- 01_RTL/ (for RTL simulation):

./01_run_vcs_rtl

./01_run_vcs_rtl_CG

- 02_SYN/ (for Synthesis):

./clk_dc_shell (for CG)

./01_run_dc_shell

(Check the design which contains **Latch** and **Error** or not in **syn.log**)

(Check the design's timing in /Report/SUDOKU.timing to see if the slack is **MET**)

- 03_GATE_SIM/ (Gate Level simulation):

./01_run_vcs_gate

- `./01_run_vcs_gate_CG`
 - 04_PTPX/ (PrimeTime power analysis):
 - `./01_run_ptpx`
 - `./01_run_cg_ptpx`
 - JG/ (JasperGold SEC)
 - `./01_run`
 - `./02_run`
 - You can key in `./09_clean_up` to clear all log files and dump files in each folder
 - 09_SUBMIT/ (submit your files):
 - `./00_tar`
 - `./01_submit 1st_demo/2nd_demo`
 - `./02_check 1st_demo/2nd_demo`
3. You should make the clock period values **identical** in 00_TESTBED/PATTERN.v and 02_SYN/syn.tcl
4. You can ignore the warning about TFIPC and SDFCOM_CFTC; however, timing violations is not allowed.

Warning-[TFIPC] Too few instance port connections

```
Warning-[SDFCOM_CFTC] Cannot find timing check
SAD SYN.sdf, 680150
module: DFFRBP, "instance: TESTBED.u_SAD.determinant_reg_17_"
SDF Warning: Cannot find timing check $hold(posedge CK,posedge RB,...)
```

```
"/RAID2/COURSE/BackUp/2023_Spring/iclab/iclabta01/umc018/Verilog/umc18_neg.v", 8234: Timing violation in TESTBED.u_SUDOKU.
$setuphold( posedge CK 666 (flag == 1'b1):819000, posedge D:818882, limits: (175,-77) );
```

A careful person will eventually have good luck.

